

Phase 1: Local Folder Setup & Environment

Open your terminal or command prompt and execute the following commands to create your project structure.

```
# 1. Create the project folder
mkdir Enterprise_RAG_App
cd Enterprise_RAG_App

# 2. Create a virtual environment to keep dependencies clean
python -m venv venv

# 3. Activate the virtual environment
# On Windows:
venv\Scripts\activate
# On macOS/Linux:
source venv/bin/activate

# 4. Create the required files
touch app.py
touch requirements.txt
touch .env
touch .gitignore
```

Your folder structure should now look exactly like this:

```
Enterprise_RAG_App/
├── venv/           # Virtual Environment Folder
├── app.py          # Main Streamlit Python Application
├── requirements.txt # Package Dependencies
├── .env            # Secret Environment Variables
└── .gitignore      # Files to hide from GitHub
```

Phase 2: Configuration & Secrets Handling

1. Open `.gitignore` and add the following lines to ensure your API keys and local databases are never uploaded to GitHub:

```
.env
venv/
__pycache__/
chroma_db/
*.pdf
```

2. Open `.env` and add your Google Gemini API Key:

```
GEMINI_API_KEY=your_actual_api_key_here
```

3. Open `requirements.txt` and add the production dependencies:

```
streamlit==1.32.0
google-generativeai==0.3.0
chromadb==0.4.24
pypdf==4.1.0
langchain-text-splitters==0.0.1
python-dotenv==1.0.1
```

4. Install the dependencies:

```
pip install -r requirements.txt
```

Phase 3: The Application Code (`app.py`)

Copy the entire block below into `app.py`.

It is divided into two parts:

1. **The Trainer Shell:** Handles the UI, File Upload, and Document Chunking.
2. **The Student Lab Workspace:** Handles the Retrieval, Prompt Assembly, and LLM Generation with Graceful Failure Guardrails.

```
# =====
# ENTERPRISE RAG APPLICATION (app.py)
# Run via terminal: streamlit run app.py
# =====
```

```
import os
```

```

import streamlit as st
import chromadb
from chromadb.api.types import Documents, EmbeddingFunction, Embeddings
from pypdf import PdfReader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from dotenv import load_dotenv

# Official Google GenAI SDK
from google import genai
from google.genai import types

# Load environment variables (API Key)
load_dotenv()
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")

# Initialize Gemini Client
if not GEMINI_API_KEY:
    st.error("🚫 GEMINI_API_KEY missing. Please add it to your .env file.")
    st.stop()
client = genai.Client(api_key=GEMINI_API_KEY)

# =====
# CUSTOM EMBEDDING FUNCTION & UI SHELL
# =====

class GeminiEmbeddingFunction(EmbeddingFunction):
    """Custom ChromaDB embedding function using Google Gemini text-embedding-004"""
    def __call__(self, input: Documents) -> Embeddings:
        response = client.models.embed_content(
            model="text-embedding-004",
            contents=input
        )
        return [emb.values for emb in response.embeddings]

# Streamlit Page Config
st.set_page_config(page_title="Enterprise RAG Assistant", page_icon="📄", layout="wide")
st.title("📄 Secure Enterprise Document RAG")
st.markdown("Upload a PDF document and ask questions. The AI will cite its sources.")

# Initialize Session State Variables
if "messages" not in st.session_state:
    st.session_state.messages = []
if "collection" not in st.session_state:
    st.session_state.collection = None

# --- SIDEBAR: DOCUMENT INGESTION ---
with st.sidebar:
    st.header("📄 Document Ingestion")
    uploaded_file = st.file_uploader("Upload a PDF", type="pdf")

```

```

if uploaded_file is not None and st.session_state.collection is None:
    with st.spinner("Parsing and chunking document..."):
        # 1. Read PDF
        reader = PdfReader(uploaded_file)
        raw_text = "".join([page.extract_text() + "\n" for page in reader.pages])

        # 2. Chunk Text
        splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100)
        chunks = splitter.split_text(raw_text)

        # 3. Initialize Ephemeral ChromaDB
        chroma_client = chromadb.Client()
        collection = chroma_client.create_collection(
            name="temp_doc_db",
            embedding_function=GeminiEmbeddingFunction()
        )

        # 4. Ingest
        ids = [f"chunk_{i}" for i in range(len(chunks))]
        metadatas = [{"source": uploaded_file.name, "chunk": i} for i in range(len(chunks))]
        collection.add(documents=chunks, ids=ids, metadatas=metadatas)

        st.session_state.collection = collection
        st.success(f"✅ Indexed {len(chunks)} chunks successfully!")

# --- DISPLAY CHAT HISTORY ---
for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.markdown(msg["content"])
        if "sources" in msg:
            with st.expander("View Retrieved Sources"):
                for source in msg["sources"]:
                    st.info(source)

# =====
# 🎓 STUDENT LAB WORKSPACE: RETRIEVAL, GUARDRAILS & GENERATION
# =====

```

- **Task 1: Retrieve Context from the Vector Database (TODO 1)**

- Call the `.query()` method on `st.session_state.collection`.
- Pass the user's question (`prompt`) into the `query_texts` argument (remember, it must be a list!).
- Set `n_results=3` to retrieve the top 3 most relevant document chunks.

- *Note: The code below your query will automatically format these results into a `context_text` string and a `source_list`.*
 - **Task 2: Build the Guardrailed Prompt Template (TODO 2)**
 - Define a `system_instruction` string that forces the model to act as a strict Enterprise Q&A Assistant.
 - **Add Guardrail 1:** Explicitly tell the model to answer *ONLY* using the provided context.
 - **Add Guardrail 2:** Give it a strict escape hatch. Instruct it: *"If the answer is not contained in the context, explicitly state: 'I cannot answer this based on the provided documents.' Do NOT hallucinate."*
 - **Add Guardrail 3:** Instruct the model to cite the chunk number inline (e.g., [Chunk 1]) when making claims.
 - Combine your `context_text` and the user's `prompt` into a single string called `user_payload`.
 - **Task 3: Execute the LLM Generation (TODO 3)**
 - Use a `try...except` block to catch any API errors.
 - Call `client.models.generate_content()`.
 - Set the model to `"gemini-2.5-flash"`.
 - Pass your `user_payload` as the `contents`.
 - Configure `types.GenerateContentConfig`:
 - Pass your `system_instruction` here.
 - **Crucial RAG Step:** Set `temperature=0.0`. This makes the model deterministic and severely limits its ability to "creatively" hallucinate answers outside of your context.
 - Extract the generated text using `response.text`.
 - Display the text using `st.markdown(final_answer)` and update the chat history array (`st.session_state.messages.append`) so the UI remembers the conversation.
-

Phase 4: How to Run and Test the Application

1. In your terminal, make sure your virtual environment is active.
2. Run the Streamlit application: `streamlit run app.py`
3. A browser window will open at `http://localhost:8501`.
4. **Testing the Guardrails (Failure Analysis):**
 - *Upload a PDF* (Use any PDF, e.g., an employee handbook or a research paper).
 - **Test 1 (In-Scope):** Ask a factual question explicitly covered in the text. Ensure it cites the [Chunk].
 - **Test 2 (Out-of-Scope / Hallucination Trigger):** Ask "Who won the World Cup in 2022?".
 - *Expected Behavior:* The app should gracefully fail and output: *"I cannot answer this based on the provided documents."*
 - **Test 3 (Ambiguous):** Ask a vague question like "What is the policy?" and watch the model attempt to synthesize the top-3 chunks retrieved.

Phase 5: Preparing for Deployment & GitHub (Portfolio Submission)

To make this the ultimate portfolio piece, students should push this code to GitHub.

1. Create a `README.md` file:

Explain what the project does. This is critical for recruiters.

Secure Enterprise RAG Assistant 

An **end-to-end** Retrieval-Augmented Generation (RAG) application built **with** **Google Gemini 2.5 Flash**, **ChromaDB**, **and** **Streamlit**.

Features

Semantic Search: Uses `text-embedding-004` to retrieve highly relevant document chunks.
Anti-Hallucination Guardrails: Strict prompt engineering **with** Temperature `0.0` ensures the model refuses to answer **out-of-scope** queries.
Source Citation: Transparently displays the exact document chunks used to generate the response.
Ephemeral Vector DB: Automatic **in-memory** indexing of uploaded PDFs via `RecursiveCharacterTextSplitter`.

How to Run Locally

1. Clone the repository.
2. Run `pip install -r requirements.txt`.
3. **Create** a `.env` file **and add** `GEMINI_API_KEY=your_key`.
4. Run `streamlit run app.py`.

2. Push to GitHub:

```
git init
git add .
git commit -m "feat(rag): build fullstack streamlit RAG app with ChromaDB and Gemini"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/Enterprise_RAG_App.git
git push -u origin main
```